ＳＱＬ　主键与索引笔记

一、主键的约束本质

主键　ＰＲＩＭＡＲＹ　ＫＥＹ　由两条规则合成：ＮＯＴ　ＮＵＬＬ　加　ＵＮＩＱＵＥ，即每一行都必须有值且全表不重复。一个表只能有一个主键，但主键可以由多个列组成，这叫复合主键。主键的作用是给每一行一个唯一且稳定的标识，让更新、删除和外部引用有明确的落点。没有主键的表在并发更新和做增量同步时无法定位唯一行，容易产生重复数据。

二、代理主键与业务主键

用自增　ＩＤ　或　ＵＵＩＤ　做主键称为代理主键，与业务无关，优点是长度小、永不变化。用身份证号、手机号、订单号做主键称为自然主键，风险在于业务规则一旦变更，主键就要改，而所有引用它的表都要级联更新。实际项目中的常见做法是自增整型或雪花　ＩＤ　做物理主键，业务唯一号建唯一索引　ＵＫ，两者各司其职。

三、聚簇索引与物理存储

ＩｎｎｏＤＢ　中主键就是聚簇索引，叶子节点直接存放整行数据，所以表本身就是按主键顺序排列的一棵　Ｂ　加树。这带来两个结果：一是按主键范围查询极快，因为数据物理相邻；二是主键选得不好会引发页分裂，例如用随机　ＵＵＩＤ　做主键会让插入落在树的随机位置，不断拆分数据页，产生大量碎片并放大写入　ＩＯＰＳ。这也是推荐单调递增主键的根本原因。

四、二级索引与回表

非主键索引都是二级索引，其叶子节点存的不是整行，而是索引列值加对应的主键值。因此通过二级索引查到记录后，还要拿主键值再走一遍聚簇索引才能取到其他列，这一步叫回表。把常用查询列直接放进联合索引，可以让查询只扫索引就取到全部所需列，这叫索引覆盖，用　ｅｘｐｌａｉｎ　查看时　Ｅｘｔｒａ　显示　Ｕｓｉｎｇ　ｉｎｄｅｘ。

五、联合索引与最左前缀

联合索引按列的顺序构成一棵有序　Ｂ　加树，遵循最左前缀原则。建立　ａ、ｂ、ｃ　三列的联合索引后，能命中　ｗｈｅｒｅ　ａ　等于、ｗｈｅｒｅ　ａ　和　ｂ　同时等于、以及　ａ、ｂ、ｃ　三个都等于，也能支持以　ａ　开头的范围排序。但是单独查　ｗｈｅｒｅ　ｂ　等于用不上这个索引，因为　ｂ　只在同一　ａ　值内部有序。范围条件之后的列会失去有序性，所以　ｗｈｅｒｅ　ａ　等于　ａｎｄ　ｂ　大于　ａｎｄ　ｃ　等于里，ｃ　无法走索引定位。设计列顺序时，把等值查询列放前面，区分度高的列放前面。

六、实操建议

第一，每张表都建主键，不要依赖隐藏 _ｒｏｗｉｄ。第二，主键用　ＢＩＧＩＮＴ　自增或趋势递增的雪花　ＩＤ，避免随机　ＵＵＩＤ　直接做聚簇主键。第三，业务唯一性用唯一索引保证，而不是在应用层先查后插，后者在高并发下必然产生重复。第四，上线前用　ｅｘｐｌａｉｎ　确认索引真的被选中，关注　ｔｙｐｅ　是否为　ｒｅｆ　或　ｒａｎｇｅ，ｒｏｗｓ　估算是否合理。第五，冗余索引不但不能加速，还会拖慢写入并占用缓冲池，定期用系统表清理未使用索引。